Loop Unrolling Example 2

Loop Unrolling Example 2

Without Scheduling

With Scheduling

Clock Cycles / Time Improvement

```
for (i=999; i>=0; i=i–1)
        x[i] = x[i] + s;
```

Architecture Features

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Figure 3.2 Latencies of FP operations used in this chapter. The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0 because the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0 (which includes ALU operation to branch).

Latency

Converted to Assembly

The straightforward RISC-V code, not scheduled for the pipeline, looks like this:

```
Loop:   fld     f0,0(x1)     //f0=array element
        fadd.d  f4,f0,f2     //add scalar in f2
        fsd     f4,0(x1)     //store result
        addi    x1,x1,–8     //decrement pointer
                             //8 bytes (per DW)
        bne     x1,x2,Loop   //branch x1≠x2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for RISC-V with the latencies in Figure 3.2.

Unrolling without Schedule

4

Example Show our loop unrolled so that there are four copies of the loop body, assuming x1 − x2 (that is, the size of the array) is initially a multiple of 32, which means that the number of loop iterations is a multiple of 4. Eliminate any obviously redundant computations and do not reuse any of the registers.

Answer Here is the result after merging the addi instructions and dropping the unnecessary bne operations that are duplicated during unrolling. Note that x2 must now be set so that Regs[x2]+32 is the starting address of the last four elements.

```
Loop:  fld     f0,0(x1)
       fadd.d  f4,f0,f2
       fsd     f4,0(x1)       //drop addi & bne
       fld     f6,-8(x1)
       fadd.d  f8,f6,f2
       fsd     f8,-8(x1)      //drop addi & bne
       fld     f0,-16(x1)
       fadd.d  f12,f0,f2
       fsd     f12,-16(x1)    //drop addi & bne
       fld     f14,-24(x1)
       fadd.d  f16,f14,f2
       fsd     f16,-24(x1)
       addi    x1,x1,-32
       bne     x1,x2,Loop
```

Bne moved to end
Of unrolled code

```
for (i=999; i>=0; i=i-1)
       x[i] = x[i] + s;
```

The straightforward RISC-V code, not scheduled for the pipeline, looks like this:

```
Loop:  fld     f0,0(x1)     //f0=array element
       fadd.d  f4,f0,f2     //add scalar in f2
       fsd     f4,0(x1)     //store result
       addi    x1,x1,-8     //decrement pointer
                            //8 bytes (per DW)
       bne     x1,x2,Loop   //branch x1≠x2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for RISC-V with the latencies in Figure 3.2.

We have eliminated three branches and three decrements of x1. The addresses on the loads and stores have been compensated to allow the addi instructions on x1 to be merged. This optimization may seem trivial, but it is not; it requires symbolic substitution and simplification. Symbolic substitution and simplification will rearrange expressions so as to allow constants to be collapsed, allowing an expression such as $((i+1)+1)$ to be rewritten as $(i+(1+1))$ and then simplified to $(i+2)$. We will see more general forms of these optimizations that eliminate dependent computations in Appendix H.

Without scheduling, every FP load or operation in the unrolled loop is followed by a dependent operation and thus will cause a stall. This unrolled loop will run in 26 clock cycles—each fld has 1 stall, each fadd.d has 2, plus 14 instruction issue cycles—or 6.5 clock cycles for each of the four elements, but it can be scheduled to improve performance significantly. Loop unrolling is normally done early in the compilation process so that redundant computations can be exposed and eliminated by the optimizer.

```
for (i=999; i>=0; i=i-1)
        x[i] = x[i] + s;
```

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

The straightforward RISC-V code, not scheduled for the pipeline, looks like this:

```
Loop:   fld      f0,0(x1)      //f0=array element
        fadd.d   f4,f0,f2      //add scalar in f2
        fsd      f4,0(x1)      //store result
        addi     x1,x1,-8      //decrement pointer
                               //8 bytes (per DW)
        bne      x1,x2,Loop    //branch x1≠x2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for RISC-V with the latencies in Figure 3.2.

Unrolling with Schedule

Avoid Hazard

Example Show how the loop would look on RISC-V, both scheduled and unscheduled, including any stalls or idle clock cycles. Schedule for delays from floating-point operations.

Answer Without any scheduling, the loop will execute as follows, taking nine cycles:

```
                                          Clock cycle issued
Loop:   fld      f0,0(x1)                 1
        stall                             2
        fadd.d   f4,f0,f2                 3
        stall                             4
        stall                             5
        fsd      f4,0(x1)                 6
        addi     x1,x1,-8                 7
        bne      x1,x2,Loop               8
```

We can schedule the loop to obtain only two stalls and reduce the time to seven cycles:

```
Loop:   fld      f0,0(x1)
        addi     x1,x1,-8
        fadd.d   f4,f0,f2
        stall
        stall
        fsd      f4,8(x1)
        bne      x1,x2,Loop
```

The stalls after `fadd.d` are for use by the `fsd`, and repositioning the `addi` prevents the stall after the `fld`.

Reordering to Avoid Hazard

```
for (i=999; i>=0; i=i-1)
    x[i] = x[i] + s;
```

Example Show the unrolled loop in the previous example after it has been scheduled for the pipeline with the latencies in Figure 3.2.

Answer

```
Loop:  fld     f0,0(x1)
       fld     f6,-8(x1)
       fld     f0,-16(x1)
       fld     f14,-24(x1)
       fadd.d  f4,f0,f2
       fadd.d  f8,f6,f2
       fadd.d  f12,f0,f2
       fadd.d  f16,f14,f2
       fsd     f4,0(x1)
       fsd     f8,-8(x1)
       fsd     f12,16(x1)
       fsd     f16,8(x1)
       addi    x1,x1,-32
       bne     x1,x2,Loop
```



The execution time of the unrolled loop has dropped to a total of 14 clock cycles, or 3.5 clock cycles per element, compared with 8 cycles per element before any unrolling or scheduling and 6.5 cycles when unrolled but not scheduled.

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

The straightforward RISC-V code, not scheduled for the pipeline, looks like this:

```
Loop:  fld     f0,0(x1)     //f0=array element
       fadd.d  f4,f0,f2     //add scalar in f2
       fsd     f4,0(x1)     //store result
       addi    x1,x1,-8     //decrement pointer
                            //8 bytes (per DW)
       bne     x1,x2,Loop   //branch x1≠x2
```

Let's start by seeing how well this loop will run when it is scheduled on a simple pipeline for RISC-V with the latencies in Figure 3.2.